

Webflux

¿Qué es Spring WebFlux y qué problema resuelve?

Es el motor reactivo de Spring, lo diseñaron para resolver el problema de la concurrencia masiva y el desperdicio de recursos. En los modelos más tradicionales y bloqueantes (Spring MVC), el servidor le asigna un hilo a cada petición y si consultamos una base de datos lenta, ese hilo se queda congelado esperando. Si llegan miles de usuarios, nos vamos a quedar sin hilos y nuestro servidor va a colapsar.

WebFlux nos ayuda a resolver esto usando un Event Loop (Netty). Funciona de manera asíncrona y no bloquea, cuando hay una espera, se libera el hilo para despachar a otros usuarios, y vuelve a la respuesta cuando el dato está listo.

A la larga se pueden usar dos tipos de flujos (conocidos como Publishers):

Mono: Representa una receta que emitirá cero o un elemento (por ejemplo, buscar a un empleado específico por su ID).

Flux: Representa una receta que emitirá de cero a N elementos, y puede ser un flujo infinito (por ejemplo, lecturas constantes de un sensor aquí no solo una).

¿Qué significa que sea Lazy (perezoso)?

Declarar un Mono o un Flux no ejecuta nada, solo nos crea la "receta" de lo que debe pasar. Los datos no comienzan a fluir ni los procesos a ejecutarse hasta que alguien se suscribe al flujo (llamando al método `.subscribe()`). En nuestras APIs, quien hace esta suscripción por nosotros es el framework de Spring cuando tiene que devolverle la respuesta al cliente por HTTP.

¿Dónde se ve en mi código?

Implementé ambos conceptos separados en dos proyectos para demostrar su naturaleza:

Mono: En 01-webflux-mono/src/.../EmployeeRepository.java, el método `findById` devuelve un `Mono<Employee>`. Para simular el problema, agregué `.delayElement(LATENCIA)` que obliga al flujo a esperar 5 segundos antes de emitir el empleado.

Flux: En 02-webflux-flux/src/.../SensorService.java, generé un flujo infinito de datos simulando un sensor de temperatura usando `Flux.interval(CADENCIA)` y así creamos una lectura cada segundo.

Stream de Eventos: En `LecturaRestController.java`, el endpoint `/stream` devuelve el Flux utilizando `produces = MediaType.TEXT_EVENT_STREAM_VALUE`. Esto le dice al servidor que no espere a tener todos los datos (como lo haría un JSON normal que forma una lista), sino que empuje cada lectura por la conexión abierta apenas el Flux la emita.

Qué pasa si no lo uso (y el experimento de los 5 segundos)

Para demostrar la verdadera diferencia, en el repositorio del proyecto 01 (EmployeeRepository.java) está el método trampa llamado findByIdBloqueante() que hace exactamente lo mismo, pero usando un tradicional Thread.sleep(5000) para dormir el hilo de Java. Cuando ejecutamos el script de estrés comparar.sh y lanzar 50 peticiones simultáneas:

El modelo bloqueante: Como mi procesador tiene un número limitado de núcleos, el servidor se queda sin hilos eventualmente. Las peticiones empiezan a hacer cola esperando a que se libere un hilo. Aunque el dato tarda 5 segundos, la última petición tarda mucho más en resolverse porque tiene que esperar su turno.

El modelo reactivo: Al golpear el endpoint que devuelve el Mono, mi servidor libera los hilos durante los 5 segundos de espera. Las 50 peticiones se procesan al mismo tiempo y todas las respuestas regresan en exactamente 5 segundos, no importa que sean 50 o 500.

¿En qué casos WebFlux no vale la pena? WebFlux solo funciona si todo el ecosistema es reactivo. Si usamos controladores con Mono y Flux, pero por debajo conectamos la base de datos usando el driver tradicional bloqueante de MySQL/JDBC, o hacemos procesamiento intensivo de CPU (como encriptar archivos gigantes), el hilo de Netty se bloqueará. En esos casos, solo vamos a agregar una curva de aprendizaje en el código sin ganar nada de rendimiento. Necesitamos usar drivers que sean reactivos (como R2DBC) para que valga la pena.

Cómo correrlo

Para el Mono en terminal:

Levanta el proyecto: ./mvnw spring-boot:run

En otra terminal corremos el script de comparación: ./scripts/comparar.sh 50 (vemos cómo el reactivo se vence).

Para el Flux (El flujo infinito):

En la carpeta 02-webflux-flux, levanta el proyecto: ./mvnw spring-boot:run

Consume el Server-Sent Event sin buffer: curl -N http://localhost:8075/api/lecturas/stream (la terminal imprime una lectura viva cada segundo).